

NRLC-03: Dashboard Migration

Series: NRLC — New Relic to Dynatrace Migration Deep Dives | **Notebook:** 3 of 9 |
Created: April 2026 | **Last Updated:** 10/05/2026

Overview

Dashboards are the highest-volume migration artifact and the most user-visible. This deep dive covers the structural differences between NR dashboards and DT documents, the conversion behavior of the `DashboardTransformer`, the widget-level translation pipeline, the gaps you should plan for (variables, dynamic content), and the validation pattern.

Phase 19 widget parity — funnel composites (`countIf` per stage + `funnelEmulation: true`), native honeycomb for heatmap, event-feed table with canonical sort + `eventFeedMode`, cascading variables with `dependsOn`, dashboard permissions (public read / public read-write / private), and saved filter sets via `savedViews` all land automatically via `migrate.py --components dashboards`. **Phase 24** added `saved_filter_notebook_transformer` for NR Data Apps → DT `type=='notebook'` Documents (markdown + DQL cells).

Table of Contents

1. [NR Dashboard Model](#)
 2. [DT Dashboard Model \(Documents v2\)](#)
 3. [DashboardTransformer Behavior](#)
 4. [Widget-Level Translation](#)
 5. [Multi-Page Dashboards](#)
 6. [Variables & Dynamic Content](#)
 7. [Layout Preservation](#)
 8. [Validation Strategy](#)
-

Prerequisites

Requirement	Details
Audience	Engineers migrating dashboards; UX reviewers comparing NR ↔ DT visualizations
Standalone	This notebook is self-contained for dashboard migration. No required prerequisite reading.
Optional depth	NRLC-02 (full NRQL → DQL compiler), NRLC-08 (validation reference), NRLC-09 (toolchain reference)
Tooling	<code>Dynatrace-NewRelic DashboardTransformer</code> (Python) or <code>nrql-engine DashboardTransformer</code> (TypeScript)

Embedded Translation Context — Dashboard Query Patterns

Every dashboard widget contains a query. These are the NRQL → DQL patterns that show up in 95% of dashboard widgets — enough to migrate dashboards without consulting the full NRLC-02 compiler reference.

Simple aggregation

```
-- NRQL
SELECT count(*), average(duration) FROM Transaction
WHERE appName = 'checkout' SINCE 1 hour ago FACET host
```

```
fetch spans, from:-1h
| filter dt.service.name == "checkout" and isNotNull(endpoint.name)
| summarize {count = count(), avg_dur = avg(duration)}, by:{host.name}
```

Every `FROM Transaction` translation needs both filters: `isNotNull(endpoint.name)` keeps request root spans only (one per NR transaction — without it `fetch spans` also counts database, outbound and internal spans), and `dt.service.name` is the Dynatrace service name (`service.name` is the OpenTelemetry attribute and misses spans that do not carry it). See NRLC-02 §6.

Time series widget

```
-- NRQL
SELECT count(*) FROM Transaction TIMESERIES 5 minutes
```

```
fetch spans, from:-1h
| filter isNotNull(endpoint.name)
| makeTimeseries count = count(), interval:5m
```

Percentage widget

```
-- NRQL
SELECT percentage(count(*), WHERE httpResponseCode = 200) FROM Transaction
```

```
fetch spans, from:-1h
| filter isNotNull(endpoint.name) and isNotNull(http.response.status_code)
| summarize success_pct = 100.0 * countIf(http.response.status_code == 200) /
count()
```

Map NR `httpResponseCode` to `http.response.status_code`. `http.status_code` is not a Dynatrace span field; a widget built on it runs and shows a near-zero percentage.

COMPARE WITH (week-over-week)

```
-- NRQL
SELECT count(*) FROM Transaction COMPARE WITH 1 week ago SINCE 1 day ago
```

```
fetch spans, from:-1d
| filter isNotNull(endpoint.name)
| summarize current = count()
| append [
  fetch spans, from:-192h, to:-168h
  | filter isNotNull(endpoint.name)
  | summarize previous = count()
]
```

Translation Confidence

Widget pattern	Confidence	Notes
Simple count / average / sum	HIGH	Direct mapping
Aggregation with FACET	HIGH	<code>by:{...}</code> syntax
TIMESERIES with interval	HIGH	<code>makeTimeseries interval:Nm</code>
Percentage / filter aggregation	HIGH	<code>countIf(condition)</code>
COMPARE WITH	MEDIUM	Verify time alignment after migration
Subquery with <code>lookup</code>	MEDIUM	Check looked-up data exists
<code>apdex()</code> widget	LOW	Configure DT Apdex separately
<code>funnel()</code> widget	LOW	Manual reformulation

1. NR Dashboard Model

An NR dashboard is a hierarchical structure:

```
Dashboard
├─ Page (multiple per dashboard)
│   └─ Widget
│       ├── visualization (line, bar, table, billboard, ...)
│       ├── NRQL query
│       └─ layout (row, column, width, height)
└─ ...
└─ permissions (per dashboard)
```

Key fields exposed via NerdGraph (`DashboardEntity`):

- `name` , `description` , `permissions`
- `pages[]` — each with `name` , `widgets[]`
- `widgets[].rawConfiguration` — JSON containing `nrqlqueries[]` , `colors` , `axis` , `thresholds` , etc.
- `widgets[].title` , `widgets[].layout`
- `widgets[].visualization.id` — identifies widget type

2. DT Dashboard Model (Documents v2)

Dynatrace Gen3 dashboards are stored as **Documents** in the Documents v2 API. Each dashboard is a single-page JSON document:

```
{
  "version": 13,
  "variables": [],
  "tiles": {
    "<tile-id>": {
      "type": "data",
      "title": "...",
      "query": "fetch logs | filter ...",
      "visualization": "lineChart",
      "visualizationSettings": { ... }
    }
  },
  "layouts": {
    "<tile-id>": { "x": 0, "y": 0, "w": 8, "h": 6 }
  }
}
```

Critical structural differences from NR:

Aspect	NR	DT
Pages	Multiple per dashboard	One per document (multi-page → multiple documents)
Query language	NRQL	DQL
Layout grid	Row-based, fixed widths	Free grid (<code>x</code> , <code>y</code> , <code>w</code> , <code>h</code>)
Widget identity	UUIDs in raw config	Tile IDs in <code>tiles</code> map
Storage	Account-scoped	Document with sharing settings (private/shared/public)

3. DashboardTransformer Behavior

The transformer (in both `Dynatrace-NewRelic/transformers/dashboard_transformer.py` and `nrql-engine/src/transformers/dashboard.ts`) handles the full conversion pipeline:

```
NR Dashboard JSON
↓
for each page:
  ↓
  [Convert page name → document name]
  ↓
  for each widget:
    [Translate NRQL → DQL via NRQLCompiler]
    [Map widget.visualization.id → DT visualization type]
    [Convert layout (row/col) → (x, y, w, h)]
    [Convert thresholds, colors, axis settings]
  ↓
  Emit one DT Document per page
```

Output: N documents per source dashboard (where N = number of NR pages). Each document has:

- A name like `<dashboard-name> - <page-name>` (avoids name collisions)
- Tags: `migrated-from-nr`, `nr-dashboard-guid:<guid>`, `nr-page:<page-name>`
- A migration-metadata tile (optional) with the source GUID for traceability

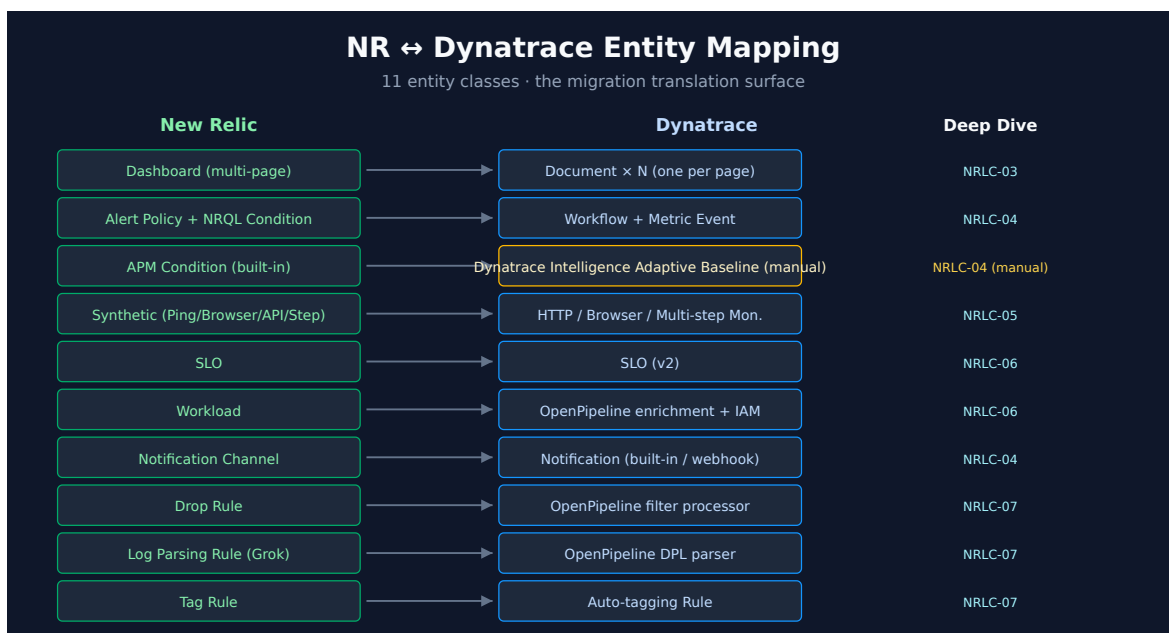
4. Widget-Level Translation

NR Widget Type	DT Visualization	Phase 19 Notes
<code>viz.line</code>	<code>lineChart</code>	direct
<code>viz.area</code>	<code>areaChart</code>	direct
<code>viz.bar</code>	<code>barChart</code>	direct
<code>viz.pie</code>	<code>pieChart</code>	direct
<code>viz.table</code>	<code>table</code>	direct
<code>viz.billboard</code>	<code>singleValue</code>	direct

NR Widget Type	DT Visualization	Phase 19 Notes
<code>viz.heatmap</code>	<code>honeycomb</code> (native)	Phase 19 — upgraded from table fallback
<code>viz.histogram</code>	<code>histogram</code>	direct
<code>viz.markdown</code>	<code>markdown</code>	direct (no query)
<code>viz.json</code>	<code>code</code> (DQL block)	direct
<code>viz.event-feed</code>	<code>table</code> with canonical timestamp sort + <code>eventFeedMode</code>	Phase 19 — upgraded from table fallback
<code>viz.funnel</code>	<code>barChart</code> composite with <code>countIf</code> per stage + <code>funnelEmulation: true</code>	Phase 19 — upgraded from markdown placeholder
Custom Nerdpack widget	not portable	document in gap analysis / see the tool's out-of-scope list (migration utilities GitHub)

Each translation preserves the title; data thresholds (red/yellow/green) map to DT `visualizationSettings.thresholds`.

Visualization-specific settings (axis labels, color overrides, sort order) are converted where DT has an equivalent. Some NR-specific settings (e.g., `nullValues: zero` semantics) emit a translation note rather than failing silently.



5. Multi-Page Dashboards

NR's multi-page dashboards become **N separate DT documents**. There's no native multi-page concept in DT Gen3 documents.

Naming pattern (default): `<dashboard-name> – <page-name>`

Tag pattern (for grouping):

```
migrated-from-nr
nr-dashboard-guid:&lt;original-guid&gt;
nr-page-index:&lt;0,1,2,...&gt;
```

Users can create a Dynatrace **launchpad** that lists all related documents with the `nr-dashboard-guid:<x>` tag, restoring the multi-page navigation pattern.

Cross-page links

If an NR widget linked to another page, the transformer emits a markdown tile in the target document with a hyperlink (`open in DT → <document URL>`). Cross-page link metadata is preserved.

6. Variables & Dynamic Content

NR dashboards support `variables` (dropdowns, text inputs, query-driven lists). DT documents also support variables but with a different schema.

NR Variable Type	DT Variable	Conversion
<code>enum</code> (static list)	<code>csv</code> variable	direct
<code>nrql</code> (query-driven)	<code>query</code> variable	NRQL → DQL via translator
<code>string</code> (free text)	<code>text</code> variable	direct
<code>default value</code>	<code>defaultValue</code> field	direct

Gotchas:

- **Cascading variables** (variable B depends on variable A) work in DT but the dependency chain must be re-declared explicitly.

- **Multi-select variables** translate, but the DQL must use `in()` filter syntax instead of NRQL's `IN`.
- **Variable interpolation in queries** changes from `{{varName}}` (NRQL) to `$varName` or `${varName}` (DT — schema-version dependent).

The transformer handles all three patterns; complex interpolation cases emit a MEDIUM-confidence translation note.

7. Layout Preservation

NR uses a **row-based** layout (12-column grid implied). DT uses a **free grid** with explicit `x`, `y`, `w`, `h` per tile.

The transformer's layout conversion:

1. Walk NR widgets in display order
2. Maintain a `currentX` cursor and `currentRowY`
3. For each widget: emit `{x: currentX, y: currentRowY, w: widget.column, h: widget.row}`
4. Advance `currentX += w`; wrap to next row when `currentX >= 12`
5. Track tallest widget in row to advance `currentRowY` correctly

Result: visually equivalent in 90%+ of cases; the remainder needs manual touch-up (typically dashboards with nonstandard widget aspect ratios). Document this as a MEDIUM-effort review item per dashboard.

8. Validation Strategy

After migration, validate dashboards with a three-tier check:

Tier	Method	What It Catches
Tier 1: Schema	DT API rejects malformed documents, plus opening each imported dashboard in the Dashboards app	Invalid JSON, missing required fields, tiles the API stored but the app cannot render
Tier 2: Query validity	<code>DQLSyntaxValidator</code> posts each tile's DQL	Bad DQL syntax, references to nonexistent fields

Tier	Method	What It Catches
Tier 3: Visual diff	Open NR dashboard side-by-side with DT document	Layout drift, missing data, color/axis mismatches

Tier 1 needs both checks, not just the API response. A document the API *accepts* can still fail dashboard validation at render time — API acceptance means the document was stored, not that it draws. For a bulk transformer run that distinction is the whole game: a systematic defect in the transformer produces the same malformed tile in every dashboard it touches, and if the only gate is the HTTP status code, N dashboards ship unusable with N successful `2xx` responses in the log. **Sample-open imported dashboards even when every call returned 2xx** — the failure mode is silent by construction.

This got sharper with **SaaS 1.344** (published 07/27/2026, staged tenant rollout from 07/29/2026 — verify whether it has reached your tenant). Before 1.344, a dashboard failing validation still loaded and showed validation warnings, so a defect surfaced as a warning nobody read. Once 1.344 reaches your tenant it does not load at all, and Dynatrace names **API- and AI-authored** dashboards as the most affected population — which is exactly what a transformer emits. The tiered strategy below is unchanged; what changed is that a skipped Tier 1 open now produces a dead dashboard rather than a noisy one.

Recommended: run Tier 3 on a 10–20% sample of dashboards. Full visual review is rarely cost-justified; sampling catches systemic issues (transformer bugs) without exhaustive manual work. Tier 1's open-and-check is cheaper than Tier 3's side-by-side comparison, so sample it more aggressively — or exhaustively on the first transformer run against a new dashboard family.

Summary

Dashboard migration is mostly mechanical — the transformer handles widgets, layout, variables, and queries. The non-mechanical work is:

- Splitting multi-page dashboards into multiple documents (and rebuilding navigation)
- Reviewing variable cascades and interpolation
- Validating visual fidelity on a sample
- Documenting MEDIUM-confidence translations for follow-up

Continue to **NRLC-04 Alert & Notification Migration** for the higher-stakes work (alerts directly affect on-call).

Tooling for Dashboard-Only Migration

End-to-end commands for migrating only dashboards — no other components touched.

```
# 1. Inventory NR dashboards (export only — nothing is written to Dynatrace)
python3 migrate.py migrate --export-only --components dashboards --output
./dashboards-run

# 2. Translate widget queries, diff against existing DT dashboards (by name),
#    and write a conversion report — nothing is imported
python3 migrate.py migrate --dry-run --diff --report --components dashboards -
-output ./dashboards-run

# 3a. Direct API import (export → transform → import); writes ./dashboards-
run/rollback-manifest.json
python3 migrate.py migrate --components dashboards --output ./dashboards-run

# 3b. Or export the transformed config from step 2 as Terraform for GitOps
management
python3 migrate.py export-terraform --input ./dashboards-run --output ./tf-
dashboards

# 4. Rollback if needed — --rollback takes the manifest file itself
python3 migrate.py migrate --rollback ./dashboards-run/rollback-manifest.json
```

Flags checked against `migrate.py` on the tool's `main` branch (10/05/2026).

There is no `--transform-only`: `--dry-run` runs export and transform, writes `<output>/transformed/dynatrace_config.json`, and skips the import. `--diff` takes effect only on a full or dry run — with `--import-only` it is ignored, and without `--dry-run` it is computed *after* the import. `--import-only` needs `--input <dir>` and does not write a rollback manifest; a full run writes `<output>/rollback-manifest.json`. Run `python3 migrate.py migrate --help` against your checkout before relying on a flag.

Single-query translation for an ad-hoc widget:

```
nrql-translator query "SELECT count(*) FROM Transaction FACET host"
```

Batch Excel translation (for very large dashboard portfolios):

```
nrql-translator excel --in dashboards-queries.xlsx --out dashboards-dql.xlsx
```

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source [NewRelic-to-Dynatrace-Migration-Utilities](#), [nrql-engine](#) (planned future home: the [dynatrace-dma](#) Dynatrace Migration Assistant organization), and [nrql-translator](#) projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official [Dynatrace documentation](#) and [New Relic documentation](#).